

HW1, версия 2: полный вывод формул

Основа для самостоятельной рукописной работы

По `models.py`, `equations.py`, `calibrate.py`
и прогону `20260927_090211_ab1cd7d2`. 27 сентября 2026 г.

Здесь разобрана именно последовательная `SmallCNN` из HW1: шесть свёрток, без `residual`-связей. Версия 2 относится к расчётам и калибровке; архитектура сети та же. Текст можно переписывать по разделам: объяснение, формула, подстановка, результат. Исходный рукописный PDF сохраняется отдельно.

1 Обозначения и размеры тензоров

S — сторона квадратного изображения, B — число изображений в `batch`. В задании $S \geq 32$ кратно 16, B — положительное целое. Для сокращения записи положим $X = BS^2$. Вход имеет форму $B \times 3 \times S \times S$ и содержит $3X$ чисел. FP32 занимает 4 bytes; 1 MiB = 2^{20} bytes.

Работаем в `eval()` и `inference_mode()`, без градиентов. Все `Conv` имеют `bias=False`, $p = \lfloor k/2 \rfloor$, `dilation 1`; после каждой — `BatchNorm` и `ReLU inplace=True`. `cuda.benchmark=False`; оба флага `allow_tf32=False`.

Для входной стороны h , ядра k , `padding` p и шага s :

$$h_{out} = \left\lfloor \frac{h + 2p - k}{s} \right\rfloor + 1.$$

Это число допустимых позиций ядра: стартуем с 0, смещаемся на s , последняя позиция не превосходит $h + 2p - k$. Для `Conv1`:

$$\left\lfloor \frac{S + 6 - 7}{2} \right\rfloor + 1 = \left\lfloor \frac{S - 1}{2} \right\rfloor + 1 = S/2.$$

Последнее равенство верно для чётного S ; округление вниз нельзя терять. `MaxPool` далее даёт $S/4$, следующие `stride-2 Conv` — $S/8$ и $S/16$.

Операция	C_{in}	C_{out}	k	s	Выход / число FP32
Conv1-BN1-ReLU	3	32	7	2	$B \times 32 \times (S/2)^2$: $8X$
MaxPool, $p = 1$	32	32	3	2	$B \times 32 \times (S/4)^2$: $2X$
Conv2-BN2-ReLU	32	64	5	1	$B \times 64 \times (S/4)^2$: $4X$
Conv3-BN3-ReLU	64	128	3	2	$B \times 128 \times (S/8)^2$: $2X$
Conv4-BN4-ReLU	128	256	1	1	$B \times 256 \times (S/8)^2$: $4X$
Conv5-BN5-ReLU	256	256	3	2	$B \times 256 \times (S/16)^2$: X
Conv6-BN6-ReLU	256	512	1	1	$B \times 512 \times (S/16)^2$: $2X$
GlobalAvgPool	512	512	—	—	$B \times 512 \times 1 \times 1$: $512B$
Flatten	—	—	—	—	$B \times 512$: $512B$, view
Linear1-ReLU	512	256	—	—	$B \times 256$: $256B$
Linear2	256	100	—	—	$B \times 100$: $100B$

BN и ReLU не меняют форму. В `Linear` есть `bias`. `Flatten` в данной сети меняет представление формы без нового массива данных.

2 GlobalAvgPool и Flatten: как получается размер выхода

После последнего блока Conv6-BN6-ReLU имеем тензор

$$A \in \mathbb{R}^{B \times 512 \times h \times h}, \quad h = S/16.$$

Здесь $A_{b,c,u,v}$ — число в изображении b , канале c , строке u и столбце v . Для каждого изображения существует 512 отдельных карт признаков размером $h \times h$.

GlobalAvgPool усредняет пространственные позиции одного канала. В коде это `AdaptiveAvgPool2d(1)`: требуемый размер выхода по высоте и ширине явно задан как 1×1 . Для фиксированных b, c :

$$g_{b,c,1,1} = \frac{1}{h^2} \sum_{u=1}^h \sum_{v=1}^h A_{b,c,u,v}.$$

Сначала складываем все h^2 чисел карты, затем делим на их количество. Например, карта 2×2 превращается в $(A_{b,c,1,1} + A_{b,c,1,2} + A_{b,c,2,1} + A_{b,c,2,2})/4$. Каналы между собой не смешиваются, разные изображения также не смешиваются. Поэтому B и 512 сохраняются, а две пространственные оси становятся единицами:

$$B \times 512 \times (S/16) \times (S/16) \longrightarrow B \times 512 \times 1 \times 1.$$

Число значений уменьшается с $B \cdot 512 \cdot (S/16)^2 = 2BS^2$ до $B \cdot 512 \cdot 1 \cdot 1 = 512B$. Это не одна средняя величина на изображение: средних величин 512. У GAP нет обучаемых весов или bias.

Арифметика. Для одной карты из $n = h^2$ чисел нужны $n-1$ сложений и одно деление. Карт $512B$, поэтому по принятому счёту операций:

$$F_{GAP} = 512B[(h^2 - 1) + 1] = 512Bh^2 = 512B \frac{S^2}{256} = 2BS^2.$$

В нашей модели трафика читаем весь вход и записываем все средние:

$$Q_{GAP} = 4(2BS^2 + 512B) \text{ bytes}.$$

Сам результат GAP занимает $4 \cdot 512B = 2048B$ bytes FP32-данных до округления allocator; это размер результата, не пик всей сети.

Flatten объединяет оси признаков, оставляя batch отдельно. В `Flatten()` по умолчанию объединяются оси начиная с первой после batch. Произведение их размеров $512 \cdot 1 \cdot 1 = 512$:

$$B \times 512 \times 1 \times 1 \longrightarrow B \times 512, \quad z_{b,c} = g_{b,c,1,1}.$$

Значения не меняются, усреднения и умножения нет. В данной сети это view того же storage: $F_{Flatten} = 0$, модельный трафик данных $Q_{Flatten} = 0$, новый массив из $512B$ чисел не выделяется.

Пример. При $S = 224$, $B = 8$ имеем $h = 14$, $h^2 = 196$. GAP превращает $8 \times 512 \times 14 \times 14$ в $8 \times 512 \times 1 \times 1$: для каждого из $8 \cdot 512 = 4096$ результатов выполняет 195 сложений и деление. Итого $4096 \cdot 196 = 802816$ FLOPs. Flatten даёт матрицу 8×512 из тех же 4096 значений.

3 Linear и ReLU: от 512 признаков к 100 выходам

После Flatten каждое изображение описывается вектором из 512 признаков. Матрица входов $Z = (z_{b,c})$ имеет размер $B \times 512$. Размеры 256 и 100 заданы архитектурой: это числа выходов соответствующих Linear, а не результат формулы stride/padding.

Linear1: Linear(512,256). Есть матрица весов $W^{(1)} \in \mathbb{R}^{256 \times 512}$ и вектор bias $\beta^{(1)} \in \mathbb{R}^{256}$. Для изображения b и выхода j :

$$u_{b,j} = \sum_{c=1}^{512} W_{j,c}^{(1)} z_{b,c} + \beta_j^{(1)}, \quad j = 1, \dots, 256.$$

Каждый выход использует все 512 входных признаков со своими весами. В матричной форме $U = Z(W^{(1)})^T + \beta^{(1)}$, где bias прибавляется к каждой строке. Формы матриц:

$$(B \times 512)(512 \times 256) \longrightarrow B \times 256.$$

Выход содержит $256B$ чисел. Параметров $256 \cdot 512 + 256 = 131328$, они общие для всех изображений и не умножаются на B . По соглашению $1 MAC = 2 FLOPs$ и с отдельным сложением bias:

$$F_{L1} = B \cdot 256 (2 \cdot 512 + 1) = 262400B.$$

ReLU после Linear1. Для каждого элемента:

$$r_{b,j} = \max(0, u_{b,j}), \quad R \in \mathbb{R}^{B \times 256}.$$

Отрицательные значения заменяются нулём, остальные сохраняются. Форма $B \times 256$ не меняется. Сравнения исключены из нашего FLOPs, поэтому $F_{ReLU} = 0$. При inplace новый массив не создаётся, но чтение и запись остаются: $Q_{ReLU} = 4 \cdot 2 \cdot 256B = 2048B$ bytes в модели трафика.

Linear2: Linear(256,100). Веса $W^{(2)} \in \mathbb{R}^{100 \times 256}$, bias $\beta^{(2)} \in \mathbb{R}^{100}$:

$$y_{b,\ell} = \sum_{j=1}^{256} W_{\ell,j}^{(2)} r_{b,j} + \beta_\ell^{(2)}, \quad \ell = 1, \dots, 100.$$

Матрица выхода $Y = R(W^{(2)})^T + \beta^{(2)}$ имеет размер $B \times 100$. Всего $100B$ выходных чисел и $100 \cdot 256 + 100 = 25700$ параметров.

$$F_{L2} = B \cdot 100 (2 \cdot 256 + 1) = 51300B.$$

Это 100 оценок классов (logits) на изображение; Softmax в данной сети нет. Во всех формулах bias β отличается от обозначения batch size B .

Численная цепочка при $B = 8$. $8 \times 512 \rightarrow 8 \times 256 \rightarrow 8 \times 256 \rightarrow 8 \times 100$. Linear1: $262400 \cdot 8 = 2099200$ FLOPs; Linear2: $51300 \cdot 8 = 410400$ FLOPs. При $S = 224$ весь head вместе с GAP даёт $802816 + 2099200 + 410400 = 3312416$ FLOPs по нашему соглашению MAC.

4 FLOPs: от одного выходного числа ко всем свёрткам

Для одного выходного числа Conv используется $C_{in}k^2$ произведений входов на веса. По соглашению задания один multiply-accumulate (MAC) считается как 2 FLOPs. Поэтому стоимость одного выходного числа равна $2C_{in}k^2$, а таких чисел $BC_{out}h_{out}^2$:

$$F_{Conv} = 2BC_{out}h_{out}^2C_{in}k^2.$$

Здесь B учитывает все изображения, C_{out} — все фильтры, h_{out}^2 — все позиции. Веса одинаковы для разных позиций и изображений, но арифметические операции повторяются. Padding не вычитаем из стандартного плотного подсчёта: считаем и позиции с дополненными нулями.

Conv1: 7×7 , $3 \rightarrow 32$, **выход** $S/2$.

$$C_{in}k^2 = 3 \cdot 49 = 147, \quad N_{out} = 8X, \quad F_1 = 2 \cdot 147 \cdot 8X = 2352X.$$

Conv2: 5×5 , $32 \rightarrow 64$, **выход** $S/4$.

$$C_{in}k^2 = 32 \cdot 25 = 800, \quad N_{out} = 4X, \quad F_2 = 2 \cdot 800 \cdot 4X = 6400X.$$

Conv3: 3×3 , $64 \rightarrow 128$, **выход** $S/8$.

$$C_{in}k^2 = 64 \cdot 9 = 576, \quad N_{out} = 2X, \quad F_3 = 2 \cdot 576 \cdot 2X = 2304X.$$

Conv4: 1×1 , $128 \rightarrow 256$, **выход** $S/8$.

$$C_{in}k^2 = 128, \quad N_{out} = 4X, \quad F_4 = 2 \cdot 128 \cdot 4X = 1024X.$$

Conv5: 3×3 , $256 \rightarrow 256$, **выход** $S/16$.

$$C_{in}k^2 = 256 \cdot 9 = 2304, \quad N_{out} = X, \quad F_5 = 2 \cdot 2304 \cdot X = 4608X.$$

Conv6: 1×1 , $256 \rightarrow 512$, **выход** $S/16$.

$$C_{in}k^2 = 256, \quad N_{out} = 2X, \quad F_6 = 2 \cdot 256 \cdot 2X = 1024X.$$

Суммируем все свёртки:

$$F_{Conv, total} = (2352 + 6400 + 2304 + 1024 + 4608 + 1024)X = 17712X.$$

Пояснение. Это точный результат принятого соглашения $1MAC = 2FLOPs$, а не число машинных инструкций GPU. Если считать скалярное произведение как n умножений и $n - 1$ сложений, получится другой счёт; в этом задании последовательно используем именно соглашение MAC.

5 FLOPs: BatchNorm, pooling, head и итог

В режиме eval BatchNorm использует сохранённые μ, σ^2 :

$$y = \gamma \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta = ax + b, \quad a = \frac{\gamma}{\sqrt{\sigma^2 + \varepsilon}}, \quad b = \beta - a\mu.$$

В нашей модели считаем 1 умножение и 1 сложение на элемент, то есть $F_{BN} = 2N_{out}$. Подготовку a, b и особенности реализации реализации kernel не считаем. Это соглашение `flops()`, а не утверждение, что PyTorch заранее слил Conv и BN в один оператор.

	BN1	BN2	BN3	BN4	BN5	BN6	Сумма
N_{out}/X	8	4	2	4	1	2	21
F_{BN}/X	16	8	4	8	2	4	42

ReLU и MaxPool. Они сравнивают значения. Сравнения не включены в наш счёт FLOPs, поэтому их вклад здесь нулевой. Это не означает нулевое время или отсутствие обращений к памяти.

GlobalAvgPool. В одном канале $n = (S/16)^2$ чисел. Чтобы получить среднее, нужны $n - 1$ сложений и 1 деление: всего n операций. Каналов 512, изображений B :

$$F_{GAP} = 512B \left(\frac{S}{16} \right)^2 = 2X.$$

Flatten не выполняет арифметику над значениями.

Linear1. Для каждого из 256 выходов — 512 MAC и одно сложение bias:

$$F_{L1} = B(2 \cdot 512 \cdot 256 + 256) = 262400B.$$

Linear2. Для каждого из 100 выходов — 256 MAC и bias:

$$F_{L2} = B(2 \cdot 256 \cdot 100 + 100) = 51300B.$$

ReLU между Linear также не добавляет FLOPs по принятому соглашению.

Общая формула одного forward всего batch:

$$\begin{aligned} F(S, B) &= F_{Conv, total} + F_{BN, total} + F_{GAP} + F_{L1} + F_{L2} \\ &= 17712BS^2 + 42BS^2 + 2BS^2 + 262400B + 51300B, \end{aligned}$$

$$\boxed{F(S, B) = 17756BS^2 + 313700B \quad \text{FLOPs.}}$$

На одно изображение делим на B ; для GFLOPs делим на 10^9 .

Пояснение. Profiler оценивает прежде всего Conv и Linear. При подсчёте только их MAC базовая сумма равна $17712BS^2 + 313344B$. Разница с нашей формулой составляет $44BS^2 + 356B$ (BN, GAP и bias). Поэтому небольшое расхождение с `flops_profiler` ожидаемо и не исправляется подбором коэффициента.

6 Память модели: каждый параметр и buffer

Нужно различать **хранимые данные модели**, **пиковую память forward** и **суммарный трафик**. Сначала найдём постоянную память модели W . Она не умножается на B : веса общие для всего batch.

Для Conv без bias число весов $K_i = C_{in,i}C_{out,i}k_i^2$. У BN на C каналов есть четыре FP32-вектора: обучаемые γ, β и buffers running mean, running variance. Ещё есть отдельный int64-счётчик num_batches_tracked размером 8 bytes. Он хранится и в eval, хотя статистика при forward не обновляется.

Блок	Веса Conv K_i	BN: FP32-чисел $4C$	Всего bytes: $4K_i + 16C + 8$
Conv1+BN1	$3 \cdot 32 \cdot 49 = 4704$	128	19 336
Conv2+BN2	$32 \cdot 64 \cdot 25 = 51200$	256	205 832
Conv3+BN3	$64 \cdot 128 \cdot 9 = 73728$	512	296 968
Conv4+BN4	$128 \cdot 256 = 32768$	1024	135 176
Conv5+BN5	$256 \cdot 256 \cdot 9 = 589824$	1024	2 363 400
Conv6+BN6	$256 \cdot 512 = 131072$	2048	532 488
Сумма	883 296	4992	3 553 200

Linear1 хранит $512 \cdot 256 + 256 = 131328$ FP32-чисел: 525312 bytes. Linear2 хранит $256 \cdot 100 + 100 = 25700$ чисел: 102800 bytes. У pooling, ReLU и Flatten параметров нет.

$$W = 4(883296 + 4992 + 131328 + 25700) + 6 \cdot 8 = 4181312 \text{ bytes.}$$

Почему v2 использует W_{alloc} , а не просто W ? В модели native CUDA allocator запрос отдельного тензора округляем вверх до 512 bytes [2]. Для размера n bytes:

$$r(n) = 512 \left\lceil \frac{n}{512} \right\rceil, \quad W_{alloc} = \sum_{\text{отдельные тензоры } j} r(n_j).$$

Нельзя сначала сложить размеры, а затем округлить один общий результат. Например, в BN1 каждый из четырёх векторов имеет $32 \cdot 4 = 128$ bytes, каждый округляется до 512; счётчик 8 тоже до 512. Вес Conv1: $18816 \rightarrow 18944$. Блок1: $18944 + 4 \cdot 512 + 512 = 21504$.

Блок	1	2	3	4	5	6	Linear1	Linear2
W_{alloc}	21504	207360	297472	135680	2363904	532992	525312	102912

$$W_{alloc} = 4187136 \text{ bytes,} \quad W_{alloc} - W = 5824 \text{ bytes.}$$

Это расчёт по формам тензоров при стандартном округлении. Повторное использование более крупных blocks и нестандартные настройки allocator могут увеличить фактически занятую память сверх этой оценки.

7 Пик активаций: почему получается $76BS^2$

Метрика задания — `torch.cuda.max_memory_allocated()`. Она отвечает на вопрос: сколько памяти через allocator было занято *одновременно* в самый дорогой момент? Складывать выходы всех слоёв нельзя: в inference старые промежуточные тензоры обычно освобождаются.

Вход x остаётся живым во внешнем коде измерения на протяжении всего forward: $3X$ чисел. Для обычного оператора в момент создания выхода одновременно живы его вход и выход. Для BN это два разных массива; для inplace ReLU выход использует тот же storage. В таблице пока учитываем только основные FP32-тензоры, без библиотечных временных buffers и постоянной памяти модели.

Момент	Живых FP32-чисел	Почему
Conv1	$11X$	$3X + 8X$: исходный вход считаем один раз
BN1	$19X$	$3X + 8X + 8X$
ReLU1	$11X$	$3X + 8X$, inplace
MaxPool	$13X$	$3X + 8X + 2X$
Conv2	$9X$	$3X + 2X + 4X$
BN2	$11X$	$3X + 4X + 4X$
Conv3	$9X$	$3X + 4X + 2X$
BN3	$7X$	$3X + 2X + 2X$
Conv4	$9X$	$3X + 2X + 4X$
BN4	$11X$	$3X + 4X + 4X$
Conv5	$8X$	$3X + 4X + X$
BN5	$5X$	$3X + X + X$
Conv6	$6X$	$3X + X + 2X$
BN6	$7X$	$3X + 2X + 2X$

ReLU2–6 не создают новую копию активации, поэтому пика соответствующего BN не превышают.

Проверка head. Даже если одновременно сохранить исходный вход ($3X$), выход features ($2X$), GAP ($512B$), Linear1 ($256B$) и Linear2 ($100B$), получится лишь $5X + 868B$. Это заведомо щедрая верхняя оценка основных тензоров head; view Flatten и inplace ReLU не добавляем. При $S \geq 32$:

$$5X + 868B = \left(5 + \frac{868}{S^2}\right) X \leq 5.848X < 19X.$$

Поэтому максимум *этой модели активаций* достигается на BN1:

$$M_{activ} = 4 \max_t N_{live}(t) = 4 \cdot 19X = 76BS^2 \text{ bytes.}$$

В развёрнутом виде: $12BS^2$ bytes исходного входа плюс два массива по $32BS^2$ bytes на BN1. При S кратном 16 все три размера кратны 512.

Пояснение. Таблица не описывает скрытые workspace, служебные тензоры kernels или возможные индексы pooling. Она доказывает максимум для выбранной модели основных активаций, а не точный peak любого CUDA backend.

8 Память v2: постоянный workspace и границы точности

Одних весов и основных активаций недостаточно. После прогрева Linear библиотека cuBLAS может сохранять рабочий buffer, выделенный через CUDA allocator PyTorch. В реализации PyTorch 2.11 стандартный размер для T4 равен [1]:

$$W_{BLAS} = (2 \cdot 4096 + 8 \cdot 16) \cdot 1024 = 8519680 \text{ bytes} = 8.125 \text{ MiB}.$$

Это сумма двух blocks по 4096 KiB и восьми по 16 KiB; она взята из `parseChosenWorkspaceSize()`, а не подобрана по измеренным ошибкам.

Почему он прибавляется к раннему пику BN1, хотя Linear стоит в конце? Сначала выполняются три прогревочных forward. Buffer, созданный при Linear на прогреве, остаётся живым. Затем сбрасывается только счётчик peak; `reset_peak_memory_stats()` не освобождает живые allocations. На следующем forward этот buffer уже есть во время BN1.

Таким образом, текущая функция `memory()` задаёт:

$$\begin{aligned} M(S, B) &= \underbrace{W_{alloc}}_{\text{модель}} + \underbrace{W_{BLAS}}_{\text{workspace}} + \underbrace{M_{activ}}_{\text{живые активации}}, \\ &= 4187136 + 8519680 + 76BS^2, \\ \boxed{M(S, B)} &= 12706816 + 76BS^2 \text{ bytes.} \end{aligned}$$

Первые два слагаемых постоянны: их сумма 12706816 bytes. Третье слагаемое зависит от batch и размера изображения: $76BS^2$ bytes. Ни один коэффициент в этой формуле не калибруется. В сравнении со старой формулой $M_{v1} = 4181312 + 76BS^2$ добавка составляет

$$M_{v2} - M_{v1} = 8525504 \text{ bytes}.$$

Условия применимости. Прогретая T4, FP32, стандартный native allocator, один используемый поток CUDA/рабочий handle, стандартный cuBLAS workspace, отсутствие дополнительных живых CUDA-тензоров пользователя. Не задаются overrides размеров workspace или округления allocations.

Что остаётся вне формулы. Дополнительная память kernels cuDNN, возможный отдельный cuBLASLt workspace, временные служебные тензоры и особенности повторного использования blocks. Эти расходы зависят от backend, формы и выбранного алгоритма. Поэтому математически точно вычислена *наша оценка*, но она не равна гарантированно измеренному пику.

Что не нужно просто прибавлять. CUDA context и свободные cached blocks относятся к другим расходам устройства; свободный reserved storage не является живой allocated memory. Сумма тензоров всех слоёв тоже неверна.

Оценка ООМ. Если условно доступно C bytes, формула даёт границу

$$B_{max}^{model}(S) = \left\lfloor \frac{C - 12706816}{76S^2} \right\rfloor \quad (C > 12706816).$$

Она не гарантирует отсутствие ООМ: физическая память расходуется и на то, чего нет в модели. Для записанной ёмкости устройства $C = 15637086208$ и $S = 512$ эта оценка даёт $B_{max}^{model} = 784$; экстраполяция за измеренную сетку здесь не проверена. В 132 фактических точках ООМ не было.

9 Bytes moved: расчёт трафика для каждого слоя

$Q(S, B)$ — суммарный объём чтения и записи за forward. Это не пиковая занятая память: один storage можно читать и перезаписывать много раз. В модели каждую активацию читаем/пишем один раз на оператор; веса Conv читаем один раз на forward. Повторное чтение из DRAM, cache reuse, fusion, временные buffers и служебные данные не моделируем.

Пусть A_{in}, A_{out} — числа FP32-элементов, $K = C_{in}C_{out}k^2$. До перевода в bytes получаем:

$$q_{Conv} = A_{in} + K + A_{out},$$

$$q_{BN} = A_{out} + 4C_{out} + A_{out},$$

$$q_{ReLU} = A_{out} + A_{out},$$

$$q_{block} = A_{in} + 5A_{out} + K + 4C_{out}.$$

Inplace экономит storage, но ReLU всё равно читает и записывает значения. Четыре BN-вектора читаются один раз на блок, не на каждый пиксель.

Блок	A_{in}	A_{out}	K	$4C_{out}$	q_{block}
1	$3X$	$8X$	4704	128	$43X + 4832$
2	$2X$	$4X$	51200	256	$22X + 51456$
3	$4X$	$2X$	73728	512	$14X + 74240$
4	$2X$	$4X$	32768	1024	$22X + 33792$
5	$4X$	X	589824	1024	$9X + 590848$
6	X	$2X$	131072	2048	$11X + 133120$
Сумма					$121X + 888288$

Оставшиеся операторы (всё ещё в FP32-элементах):

$$q_{MaxPool} = 8X + 2X = 10X,$$

$$q_{GAP} = 2X + 512B, \quad q_{Flatten} = 0,$$

$$q_{L1} = 512B + 512 \cdot 256 + 256B = 768B + 131072,$$

$$q_{ReLU, head} = 2 \cdot 256B = 512B,$$

$$q_{L2} = 256B + 256 \cdot 100 + 100B = 356B + 25600.$$

Суммируем и умножаем на 4 bytes/FP32:

$$Q = 4[(121 + 10 + 2)X + (512 + 768 + 512 + 356)B + 888288 + 131072 + 25600],$$

$$Q(S, B) = 532BS^2 + 8592B + 4179840 \text{ bytes.}$$

Точное соответствие коду. Текущая `bytes_moved()` опускает чтение bias Linear (всего $4(256 + 100) = 1424$ bytes за forward при чтении по одному разу). В storage и FLOPs bias учтён. Здесь сохраняем именно реализованную оценку трафика, а не выдаём её за полный аппаратный счётчик.

10 Latency: смысл каждого члена и два режима v2

Измеренное время — медиана семи forward после трёх прогревов; перед и после каждого измерения выполняется CUDA synchronization. Время одного forward складывается из накладных расходов и основной работы. Обозначим: t_0 — совокупные накладные расходы одного forward, seconds; P — эффективная скорость арифметики, FLOPs/s; R — эффективная скорость передачи данных, bytes/s.

Если выполнить F операций со скоростью P , нужно F/P seconds. Передать Q bytes со скоростью R — Q/R seconds. В roofline-приближении арифметика и обмен памятью частично перекрываются, поэтому используем максимум:

$$T = t_0 + \max\left(\frac{F}{P}, \frac{Q}{R}\right).$$

Это агрегатная модель всей последовательной сети. Она не утверждает полного перекрытия всех операторов; t_0 также не является временем одного kernel.

В v2 допускаем изменение эффективной скорости при большом batch:

$$P(B) = \begin{cases} P_1, & B \leq B_*, \\ P_2, & B > B_*, \end{cases} \quad P_2 \leq P_1.$$

Порог и необходимость второго режима выбираются по train. Это эмпирическое описание смены эффективности; по одному fit нельзя доказать, какой именно CUDA kernel или аппаратный эффект её вызвал.

Полная формула с раскрытыми F и Q :

$$T(S, B) = t_0 + \max\left(\frac{17756BS^2 + 313700B}{P(B)}, \frac{532BS^2 + 8592B + 4179840}{R}\right).$$

В последнем прогоне (коэффициенты ниже округлены для переписывания):

$$\begin{aligned} t_0 &\approx 0.000732280281 \text{ s} = 0.732280281 \text{ ms}, \\ P_1 &\approx 3.7150824604 \cdot 10^{12} \text{ FLOPs/s}, \\ P_2 &\approx 1.9394496515 \cdot 10^{12} \text{ FLOPs/s}, \\ R &\approx 1.1999933598 \cdot 10^{11} \text{ bytes/s}, \quad B_* = 64. \end{aligned}$$

Полная точность сохранена в results_v2/theta.json.

Как читать режимы. Сравниваем три времени: t_0 , $F/P(B)$, Q/R . Если больше всего t_0 , модель launch-bound; если Q/R — memory-bound; если $F/P(B)$ — compute-bound. При близких членах режим смешанный. Арифметическая интенсивность $I = F/Q$ (FLOPs/byte): между compute и traffic граница $I = P(B)/R$, если накладные расходы не доминируют.

Пояснение. В последнем прогоне модель относит 98 точек к compute-bound и 34 к launch-bound; memory-bound точек нет. Поэтому fitted R слабо определяется данными. Эти P, R нельзя интерпретировать как паспортные характеристики T4. Между измеренными $B = 64$ и $B = 95$ точное положение физической границы смены режима неизвестно.

11 Как получены коэффициенты latency

Всего 132 пары (S, B) : 11 размеров изображения и 12 batch sizes. Seed 2026 заранее выделяет 106 train и 26 validation. Из fit исключаются OOM, нечисловые и неположительные измерения. Параметры и порог выбираются по train; validation используется для итогового отчёта.

Для точки i обозначим измеренное время t_i , а $f_i = F(S_i, B_i)/10^9$, $q_i = Q(S_i, B_i)/10^9$. Вместо прямой оптимизации скоростей используем обратные скорости:

$$a = \frac{10^9}{P_1}, \quad c = \frac{10^9}{R}, \quad d = \frac{10^9}{P_2} - \frac{10^9}{P_1} \geq 0.$$

Тогда для фиксированного порога b_* :

$$\hat{t}_i = t_0 + \max(f_i(a + d \mathbf{1}_{B_i > b_*}), c q_i).$$

$\mathbf{1}$ равно 1 для большого batch и 0 иначе. Параметры $t_0, a, c, d \geq 0$; неотрицательность d задаёт $P_2 \leq P_1$. При одном режиме дополнительный параметр d отсутствует.

Целевая функция fit. Минимизируем сумму квадратов масштабированных остатков:

$$\min_{t_0, a, c, d \geq 0} \sum_{i \in \mathcal{T}} \left(\frac{\hat{t}_i - t_i}{s_i} \right)^2, \quad s_i = \max(t_i, 0.05 \text{ median}_{j \in \mathcal{T}} t_j).$$

$\mathcal{T}$ — только текущая обучающая часть. Нормировка не даёт большим временам полностью подавить малые. Нижняя граница масштаба защищает от чрезмерного веса очень маленьких значений. Это weighted least squares, а не минимизация median APE.

Выбор одного/двух режимов по cross-validation:

1. Берём baseline с одним P и кандидаты порога из train batch sizes. По обе стороны каждого порога должно оказаться не меньше 20% train-точек.
2. Делим только train на пять folds. Для каждого порога пять раз подгоняем параметры по четырём folds и предсказываем оставшийся.
3. По объединённым отложенным предсказаниям считаем mean APE: $\frac{100}{N} \sum_i |\hat{t}_i - t_i|/t_i$.
4. Выбираем лучший mean APE. Два режима принимаем, только если $CV_{two} < 0.95 CV_{one}$; при равенстве код оставляет baseline.
5. Заново подгоняем выбранную модель на всех 106 train-точках.

В каждом fit пробуем девять начальных приближений и выбираем успешное с минимальной суммой квадратов. Это уменьшает зависимость от старта, но не является доказательством глобального оптимума.

В сохранённом прогоне baseline CV mean APE 25.3538%, при $B_* = 64$ — 12.9261%. Улучшение около 49.0% относительно baseline, поэтому второй режим принят. Эти CV-ошибки не являются ошибками на 26 итоговых validation-точках.

12 Energy: мощность, работа и калибровка

Энергия — интеграл мощности по времени. Для почти равномерных отсчётов NVML в серии из N forward используем:

$$E^{meas} \approx \frac{\overline{P}_{GPU} \Delta t}{N}.$$

Здесь $\overline{P}_{GPU}$ — средняя измеренная мощность всей GPU в watts, Δt — время серии в seconds; результат в joules на forward. Это энергия всего batch, не одного изображения. Измеренная мощность уже содержит базовое потребление GPU, вычитать idle из итогового измерения нельзя.

Модель v2:

$$E(S, B) = (P_{idle} + P_{active})T(S, B) + e_0 + e_F \frac{F(S, B)}{10^9} + e_Q \frac{Q(S, B)}{10^9}.$$

P_{idle} — измеренная idle power (W); P_{active} — дополнительный подгоняемый член мощности (W); e_0 — фиксированные joules/forward; e_F — J/GFLOP, e_Q — J/GB. Здесь P_{idle}, P_{active} обозначают мощность, а $P(B)$ из latency — производительность: это разные величины.

Член $P_{active}T$ позволяет описать зависящие от длительности расходы активной GPU сверх idle. e_F и e_Q описывают дополнительные расходы, связанные с модельной работой. Это совместная регрессионная модель; по её коэффициентам нельзя строго разложить физическую энергию по узлам GPU.

Откуда берутся коэффициенты? Сначала фиксируем fitted latency. Берём P_{idle} как медиану idle measurements по допустимым train-строкам. Для каждой train-точки:

$$T_i = T(S_i, B_i; \hat{\theta}), \quad y_i = E_i^{meas} - P_{idle}T_i, \quad z_i = (1, F_i/10^9, Q_i/10^9, T_i).$$

Ищем $\beta = (e_0, e_F, e_Q, P_{active}) \geq 0$ методом NNLS:

$$\min_{\beta \geq 0} \sum_{i \in \mathcal{T}} \left(\frac{z_i \beta - y_i}{u_i} \right)^2, \quad u_i = \max(E_i^{meas}, \text{quantile}_{0.05, \mathcal{T}} E^{meas}).$$

Используем *предсказанное* T_i , чтобы итоговая функция зависела только от (S, B, θ) . Масштабы и квантиль вычисляются только по train.

В последнем прогоне:

$$P_{idle} = 30.97, \quad P_{active} \approx 27.7825483463, \quad e_0 = 0, \quad e_F \approx 0.00495013785384, \quad e_Q = 0.$$

Следовательно, для этих коэффициентов:

$$E(S, B) \approx 58.7525483463 T(S, B) + 0.00495013785384 \frac{17756BS^2 + 313700B}{10^9} \text{ J}.$$

Нулевой e_Q не означает бесплатный обмен памятью: в данных его стоимость не отделилась от остальных членов. NVML имеет ограниченную частоту обновления; серия для energy и одиночные synchronized forwards для latency могут давать разные накладные расходы.

13 Численный пример и проверка точности

Возьмём $S = 224$, $B = 8$; тогда $X = 8 \cdot 224^2 = 401408$. Всё ниже — *предсказания*, полученные подстановкой в формулы.

$$\begin{aligned} F &= 17756 \cdot 401408 + 313700 \cdot 8 \\ &= 7129910048 \text{ FLOPs} = 7.129910048 \text{ GFLOPs}, \\ Q &= 532 \cdot 401408 + 8592 \cdot 8 + 4179840 \\ &= 217797632 \text{ bytes}, \\ M &= 12706816 + 76 \cdot 401408 = 43213824 \text{ bytes} \\ &= 41.2119140625 \text{ MiB}. \end{aligned}$$

Поскольку $8 \leq 64$, используем P_1 . Из коэффициентов полной точности:

$$\begin{aligned} F/P_1 &\approx 0.001919179486 \text{ s}, \\ Q/R &\approx 0.001814990310 \text{ s}, \\ T &\approx 0.000732280281 + 0.001919179486 \\ &\approx 0.002651459767 \text{ s} = 2.651459767 \text{ ms}, \\ E &\approx 58.7525483463 \cdot 0.002651459767 + 0.00495013785384 \cdot 7.129910048 \\ &\approx 0.191074056 \text{ J}. \end{aligned}$$

Вторая контрольная точка проверяет ветку большого batch:

	$S = 32, B = 1$	$S = 224, B = 128$
F , FLOPs	18 495 844	114 078 560 768
Q , bytes	4 733 200	3 422 064 512
M , bytes	12 784 640	500 818 944
T , ms	0.771724	59.552348
E , J	0.0454323	4.0635568
Ветка	P_1	P_2

Как оценена точность. Для положительного измерения y_i : $\text{APE}_i = 100|\hat{y}_i - y_i|/y_i$. Median APE — медиана этих индивидуальных ошибок, mean APE — их среднее. Это не ошибка медианы измерений и не одна ошибка общего среднего.

Validation, 26 точек	v1 median	v2 median	v1 mean	v2 mean
FLOPs	0.25%	0.25%	0.25%	0.25%
Memory	28.90%	22.03%	35.75%	21.90%
Latency	27.83%	14.63%	28.26%	16.21%
Energy	21.00%	6.89%	24.09%	11.13%

Grid и split совпадают, но прогоны разные. Все 132 измерения памяти совпали, поэтому её улучшение связано с формулой. Сравнение latency/energy также включает вариативность GPU. Holdout уже использовался в прежних отчётах; для независимой проверки новой гипотезы нужны свежие конфигурации.

14 Итог для рукописной работы и источники

Четыре функции для этой сети, FP32, eval, inference, $S \in 16\mathbb{N}$, $S \geq 32$ и положительного целого B :

$$F(S, B) = 17756BS^2 + 313700B,$$

$$M(S, B) = 12706816 + 76BS^2,$$

$$T(S, B; \theta) = t_0 + \max(F(S, B)/P(B), Q(S, B)/R),$$

$$E(S, B; \theta_E) = (P_{idle} + P_{active})T + e_0 + e_F F/10^9 + e_Q Q/10^9.$$

Вспомогательные определения:

$$Q(S, B) = 532BS^2 + 8592B + 4179840, \quad P(B) = \begin{cases} P_1, & B \leq B_* \\ P_2, & B > B_* \end{cases}.$$

F измеряется в FLOPs, M, Q — в bytes, T — в seconds, E — в joules. Значения относятся к одному forward всего batch. Только latency и energy содержат fitted parameters. Все суммы FLOPs, storage и модельного трафика получены из размеров слоёв; workspace памяти — из настройки реализации.

Что переписать, чтобы вывод оставался полным. Сначала соглашения и таблицу форм. Затем общую формулу Conv и подстановки для шести слоёв; BN, GAP, head и сумму FLOPs. Для памяти — таблицу весов, округление allocations, максимум одновременно живых активаций и workspace. После этого — таблицу traffic, latency с двумя режимами, energy и обе целевые функции fit. В конце — один численный пример и ограничения. Страницы этой заготовки соответствуют этим шагам; при необходимости таблицы можно разнести на несколько рукописных листов.

Что обязательно сохранить в объяснениях. Разделять M и Q ; не умножать веса на batch; не считать inplace ReLU бесплатным по трафику; не называть оценку памяти точным измерением. Объяснить, почему warmup workspace присутствует уже при BN1. Указать, что $P(B)$ и R — эффективные параметры, а выбор порога выполнен внутри train. Наблюдаемое улучшение ошибки не снимает ограничений модели.

Основа и воспроизводимость. Архитектура: hw1/models.py. Формулы: hw1/equations.py. Калибровка: hw1/calibrate.py. Протокол: hw1/measure.py. Коэффициенты и ошибки: hw1/results_v2/theta.json, errors.json; сравнение: hw1/versions_comparison.json. Текущий автономный ноутбук: hw1/hw1_version_2.ipynb.

Первичные источники для констант реализации.

[1] PyTorch v2.11.0, CublasHandlePool.cpp: parseChosenWorkspaceSize(), getNewWorkspace(), setWorkspaceForHandle(). [Исходный код cuBLAS workspace](#).

[2] PyTorch v2.11.0, CUDACachingAllocator.cpp, round_size(): стандартное округление запроса. [Исходный код CUDA allocator](#).

Это печатная заготовка для понимания и переписывания. Итоговый рукописный PDF пользователь составляет самостоятельно; первоначальный hw1_handwritten.pdf этим файлом не заменён.